OOPs MASTERCLASS

Python + Interview Edition

Video transcript (21 chapters) + purane notes = ek complete, colourful guide.
Theory + Code + 250+ interview questions + Predict-the-Output + LLD + Cheat sheet

Part	Topic	Kya milega
1	Foundation	Why OOP, Class, Object, Method vs Function, Class Diagram
2	Constructor, self, Magic Methods	__init__, self ka logic, dunder methods
3	Fraction + Operator Overloading	Apna data type banana, __add__ __str__
4	Encapsulation	Private, name mangling, getter/setter, @property
5	Reference, Pass-by-Reference, Collections	Mutable/Immutable, copy, list of objects
6	Static Variables and Methods	Class vs instance var, @classmethod, @staticmethod
7	Aggregation / Composition	Has-A relationship, class diagram
8	Inheritance + super + MRO	Types, constructor inheritance, diamond problem
9	Polymorphism	Overriding, overloading, operator overloading, duck typing
10	Abstraction and Interfaces	abc, abstract vs interface, Java/C++ comparison
11	Advanced / Tricky Questions	Shallow/deep copy, __new__, slots, slicing, virtual destructor
12	SOLID + Design Patterns	5 principles + Singleton, Factory, Observer, Strategy
13	Predict the Output	Code padhke output batao - interview favourite
14	LLD Scenarios + Mini Project	Parking Lot, ATM, Library, Chess, IRCTC project
15	Rapid Fire + Cheat Sheet	1-line answers aur last-minute revision

KAISE PADHEIN: Har question ke upar Q-number hai. Pehle khud answer socho, phir padho. Dark boxes = code. Peele boxes = tricks / warnings.

PART 1

Foundation: OOP kya hai aur kyun?

Video Ch.1 - Ch.4: Importance of OOP, Class, Object, Practical

1.1 Problem jo OOP solve karta hai

Story: Ek college-LinkedIn jaisa platform bana rahe ho. Normal tareeke se (int, str, list, tuple, dict) coding karoge to 6-8 mahine baad code itna bada ho jata hai ki naya feature add karna mushkil ho jata hai - sab kuch confuse.
Solution: **Generality se Specificity** - built-in data types ki jagah apne **khud ke data types** banao jo problem ke liye bane ho (User, Post, Message). Yehi OOP ka core idea hai.

OOP ke 4 pillars + 1 idea	Ek line mein
Class / Object	Apna data type banana aur uska instance
Encapsulation	Data + methods ek jagah, aur data ko protect karna
Inheritance	Purani class se cheezein inherit karo, code repeat mat karo (DRY)
Polymorphism	Ek naam, multiple forms (overriding / overloading / operator overloading)
Abstraction	Complexity chhupao, sirf zaroori dikhao

Q1. OOP kya hai? Ek line mein.

OOP ek **coding technique** hai jisme hum real-world entities ko **objects** (data + behavior) ki tarah model karte hain, apne custom data types (classes) banakar. Full form: Object Oriented Programming.

Q2. OOP ki zaroorat kyun padi? (Why OOP?)

Bade applications mein sirf built-in types (int, str, list) se code unmanageable ho jata hai. OOP se code **modular, reusable, maintainable** banta hai, aur real-life problem ka direct mapping hota hai (Customer, ATM, Train...). Naya feature add karna aasaan hota hai.

Q3. Procedural vs Object-Oriented programming ka difference?

Procedural: functions pe focus, data aur functions alag, data globally share, security kam (C, basic Python scripts).
OOP: objects pe focus, data + functions bundled, data hiding possible, reusability (inheritance), real-world modeling (Java, C++, Python).

Q4. OOP ke 4 pillars batao.

Encapsulation (hide data), **Abstraction** (hide complexity), **Inheritance** (reuse), **Polymorphism** (many forms).

Q5. Python mein 'everything is an object' ka matlab? Proof do.

Python mein int, str, list, function, class - sab kisi na kisi class ke object hain. **type()** se class pata chalti hai, **dir()** se uske methods.

```
print(type(1))      # <class 'int'>
print(type([1,2]))  # <class 'list'>
print(type(len))    # <class 'builtin_function_or_method'>
L = [1, 2, 3]
L.append(4)         # list class ka method - OK
S = 'hello'
S.append('x')       # AttributeError: 'str' object has no attribute 'append'
```

Q6. Ye error kyun aata hai: 'list' object has no attribute 'upper'?

Kyunki **method sirf us class ke object pe chalta hai jisme wo define ho**. upper() str class mein hai, list class mein nahi. Har datatype ki class ne apne methods alag define kiye hain.

1.2 Class aur Object

Q7. Class kya hai?

Class ek **blueprint / template** hai jo batata hai ki object kaisa dikhega (data) aur kaise behave karega (methods). Class ka matlab hai apna **data type define karna**. Memory sirf blueprint ki hoti hai, actual object ki nahi.

Q8. Object kya hai?

Class ka **real instance**. Class se jitne chaaho objects ban sakte hain, har ek ka apna data hota hai. Object hamesha kisi class ka hi hota hai. Real life: Class = Car ka design, Object = tumhari Swift.

Q9. Class aur Object mein differences?

1) Class blueprint, object instance. **2)** Class ek hoti hai, objects kai. **3)** Class logical entity, object physical (memory leta hai). **4)** Class 'class' keyword se, object class ko call karke. **5)** Class banane mein zyada mehnat (saare methods likhne padte hain), object banana asaan - ek line.

Q10. Ek class ke andar kaun-kaun si 2 cheezein hoti hain?

1) Data (properties / attributes / fields) - object ke baare mein description. **2) Functions** (methods) - object ka behavior. Example Car: Data = color, engine, brand, seats | Methods = calculate_mileage(), open_airbag(), show_gps().

Q11. Python mein class aur object kaise banate hain?

'class' keyword + class name. Object: obj = ClassName().

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand       # data
        self.color = color
    def honk(self):              # behavior
        print(self.brand, 'beep beep')

c1 = Car('Tata', 'red')   # object
c2 = Car('Kia', 'blue')   # doosra object
c1.honk()
```

Q12. Naming convention kya follow karte hain?

Class name: PascalCase (har word ka pehla letter capital: *BankAccount*). **Variables aur methods: snake_case** (*check_balance*). Ek aur style camelCase hota hai (pehla letter small) - Java mein methods ke liye, Python mein nahi.

Q13. Object banane ka literal syntax kya hota hai?

Built-in classes ke liye Python shortcut deta hai: L = [1,2] andar se L = list([1,2]) hi hai. Isko **object literal** kehte hain. Apni classes ke liye ClassName() hi likhna padta hai.

Q14. Python ka object type kaise check karte ho? Aur class ke methods kaise dekhte ho?

type(obj) ya obj.__class__ se class; dir(obj) se saare attributes/methods; isinstance(obj, Cls) se check. IDLE/IDE mein obj. ke baad dot lagao to saare methods dikhte hain.

1.3 Method vs Function (Bahut puchha jata hai)

Point	Function	Method
Kahan define	Class ke bahar	Class ke andar
Kaun call kar sakta hai	Koi bhi (general)	Sirf us class ka object (obj.method())
Call syntax	len(L)	L.append(5)
Pehla argument	Normal	self (current object) automatically milta hai
Example	print(), len(), type()	list.append(), str.upper()

INTERVIEW TIP: Interview mein 'function' bolne ki galti ho jaye to bhi sochte raho: class ke andar wala = METHOD. Interviewer technical clarity dekhta hai.

Q15. Method aur function mein difference? len(L) aur L.append(5) alag kyun hain?

len() class ke bahar defined general function hai jo list ko argument leta hai. **append()** list class ke andar defined method hai - sirf list object L isko call kar sakta hai: L.append(5) (andar se L khud self ban jata hai).

Q16. Ek class ke methods ko doosri class ka object call kar sakta hai?

Nahi. Method sirf us class ke object ke liye available hai (ya inheritance se child ko). Is liye str object pe append() nahi chalta.

Q17. Kya ek class ka method same class ke dusre method ko directly call kar sakta hai?

Nahi - bina self ke nahi. Class ke andar ek method doosre ko tabhi access kar sakta hai jab object ke through jaye: self.other_method(). Yehi self ki zaroorat ka reason hai.

1.4 UML Class Diagram

Class diagram ka format (3 box ka rectangle):
Upar: **Class ka naam** | Beech: **Attributes (data)** | Neeche: **Methods**
+ = public (bahar se access) **-** = private (bahar se nahi) **#** = protected
Industry mein senior developer class diagram deta hai, tum uska code likhte ho.

```
+---------------------------+
|           ATM             |
+---------------------------+
| - pin                     |
| - balance                 |
+---------------------------+
| + __init__()   (-menu)    |
| + deposit()               |
| + withdraw()              |
| + check_balance()         |
+---------------------------+
```

Q18. Class diagram mein + aur - ka kya matlab hai?

+ public (class ke bahar se access), **-** private (sirf class ke andar), # protected (class + child classes). Python mein ye strict nahi hai, sirf convention (_x, __x).

Q19. Class diagram mein inheritance aur aggregation kaise dikhate hain?

Inheritance: child se parent ki taraf hollow (khali) triangle arrow. **Aggregation:** container class ki taraf **diamond** (khali), contained class ki taraf line. **Composition:** filled (bhara hua) diamond.

PART 2

Constructor, self aur Magic Methods

Video Ch.5 - Ch.6: Constructor, self in Python

2.1 Constructor (__init__)

Constructor ek special method hai jiske andar ka code **automatically execute** hota hai jaise hi object banta hai. Python mein iska naam **__init__** (double underscore dono taraf) hota hai.
Java/C++ mein constructor ka naam class ke naam jaisa hota hai; Python mein hamesha __init__ (galti se _init_ ya init likha to kaam nahi karega).

```
class ATM:
    def __init__(self):            # object banate hi auto-run
        self.pin = ''
        self.balance = 0
        print('Hello, ATM ready')
        self.menu()                # constructor se doosra method bhi call

    def menu(self):
        user = input('1.Create PIN 2.Deposit 3.Withdraw 4.Balance 5.Exit: ')
        if user == '1':  self.create_pin()
        elif user == '2': self.deposit()
        # ...

sbi = ATM()      # 'Hello, ATM ready' + menu apne aap
```

Q20. Constructor kya hota hai? Kab call hota hai?

Special method jiske andar ka code **object creation ke time automatically** execute hota hai. Python mein naam __init__. Hum ise manually call nahi karte.

Q21. Constructor ka kaam / use kya hai? Kya cheezein constructor mein likhte hain?

Wo cheezein jinka control **user ko nahi dena** aur jo app start hote hi honi chahiye: configuration, internet/database connection, GPS permission, initial values set karna (balance = 0), initial menu dikhana.
Example: app khulte hi internet se connect hona - iske liye user ko button nahi dete, wo constructor mein hota hai.

Q22. Constructor ko object kyun nahi call kar sakta? (sbi.__init__ nahi dikhta kya?)

Python mein sbi. likhne par saare methods dikhte hain - par constructor jaise **magic methods** user ke control ke liye nahi hote; unko Python khud sahi time pe call karta hai. Real ATM mein bhi 'withdraw' ka button hai, par 'constructor' ka button nahi - wo control user ko diya hi nahi jata.

Q23. Python mein constructor ka naam alag kyun hai Java/C++ se?

Java/C++ mein constructor ka naam = class name. Python mein hamesha __init__, chahe class ka naam kuch bhi ho. Isliye Python mein ek hi class mein naam-based multiple constructors ban nahi sakte.

Q24. Kya __init__ kuch return kar sakta hai?

Nahi - sirf None. Kuch aur return karoge to TypeError: __init__() should return None. Constructor ka kaam object *initialize* karna hai, banana nahi (banana __new__ ka kaam hai).

Q25. Python mein multiple constructors (overloaded) kaise banate hain?

Directly nahi (same naam doosre ko override kar deta hai). Workarounds: **1) default arguments** (def __init__(self, a=0, b=1)), **2) *args / **kwargs**, **3) @classmethod alternate constructors** (Fraction.from_string('3/4')).

```
class Fraction:
    def __init__(self, n=0, d=1):
        self.n, self.d = n, d
    @classmethod
    def from_str(cls, s):
        n, d = s.split('/')
        return cls(int(n), int(d))

f = Fraction.from_str('3/4')
```

Q26. Constructor ke types? (Java/C++ context)

Default (no-arg), **Parameterized**, **Copy constructor** (C++ mein explicit, Java mein manually). Constructor chaining: Java this(...) same class, super(...) parent class; C++ mein initializer list / delegating constructor.

Q27. Agar class mein __init__ nahi likha to kya hoga?

Python ka default (object class ka) constructor chal jata hai - kuch nahi karta. Object phir bhi ban jayega. Par agar tum Cls(10) argument pass karoge to TypeError aayega.

2.2 Magic / Dunder Methods

Jis method ke **aage aur peeche double underscore** (__init__, __str__, __add__) hota hai use **magic / dunder / special method** kehte hain. Ye Python ne **pehle se decide** kiye hain - tum apne marzi ke naam nahi bana sakte (__abc__ ka koi matlab nahi). Inki khasiyat: **ye object call nahi karta, ye kisi particular situation mein apne aap call hote hain**.

Magic method	Kab auto-call hota hai	Use
__init__	Object banate waqt	Constructor
__str__	print(obj) / str(obj)	User-friendly string
__repr__	Debug / repr(obj) / list ke andar	Developer string
__add__ / __sub__ / __mul__ / __truediv__	obj1 + obj2 etc.	Operator overloading
__eq__ / __lt__ / __gt__	== / < / >	Comparison
__len__	len(obj)	Length
__getitem__	obj[i]	Indexing
__call__	obj()	Object ko function banana
__del__	Object delete/garbage collect hone par	Destructor
__new__	__init__ se PEHLE	Object create karta hai
__enter__ / __exit__	with statement	Context manager

Q28. Magic methods kise kehte hain? Pehchante kaise ho?

Jinke aage aur peeche double underscore ho. Python mein pre-defined hain, hum apne naam se nahi bana sakte. Object inko directly call nahi karta - ye specific situation (object creation, print, + operator) pe automatically trigger hote hain.

Q29. Magic methods ka fayda kya hai?

Hum apni class ko **built-in types jaisa behave** karwa sakte hain: print ho sake, + lag sake, == chale, len() chale, for-loop chale. Isi se apna datatype (Fraction, Point, Matrix) ban pata hai.

Q30. __str__ aur __repr__ mein difference?

__**str**__: end-user ke liye readable (print() isse use karta hai). __**repr**__: developer ke liye unambiguous, ideally eval-able string (debugging, list print karte time). Agar __str__ nahi hai to print() __repr__ use karta hai. Dono nahi hain to <__main__.Cls object at 0x...> dikhta hai.

```
class P:
    def __str__(self):  return 'Point(user view)'
    def __repr__(self): return 'P(x=1,y=2)'
p = P()
print(p)        # Point(user view)
print([p])      # [P(x=1,y=2)]
```

2.3 self - sabse confusing, sabse important

Ek line mein: jis object ke saath *abhi* kaam ho raha hai, wahi **self** hota hai. Jo object ne method call kiya, wohi us method ka **self** ban jata hai.
Proof: constructor ke andar id(self) aur bahar id(sbi) same aate hain. Naya object (hdfc) banao to naya address aur naya self.

```
class ATM:
    def __init__(self):
        print('self ka id:', id(self))

sbi = ATM()
print('sbi ka id :', id(sbi))   # SAME address
hdfc = ATM()                    # alag address -> alag self
```

Q31. self kya hai?

Class ke current object ka reference. Jo object method call karta hai wahi us call mein self ban jata hai. Python automatically pehle argument mein bhej deta hai, isliye sbi.deposit() likhne par bhi deposit(self) ko self mil jata hai.

Q32. Python mein har method ke andar self kyun likhna padta hai? (Bahut important)

Class mein sirf data aur methods hote hain, aur dono ko **sirf us class ka object** access kar sakta hai. Even ek method doosre method ya data ko directly access nahi kar sakta. Isliye method ko pata hona chahiye ki wo kiske liye chal raha hai - ye 'kaun' = **self**. self.balance, self.menu() ke through hi class ke andar data/method milte hain.

Q33. Agar method se self hata do aur call karo to kya hoga?

TypeError: create_pin() takes 0 positional arguments but 1 was given. Kyunki obj.method() andar se Class.method(obj) hota hai - Python object ko automatically pehla argument bhejta hai, aur method ke paas use receive karne ki jagah nahi hai.

Q34. self ka naam kuch aur rakh sakte hain?

Technically haan (this, me), kyunki wo sirf pehla parameter hai. **Par kabhi mat karo** - convention hai aur sab developers self hi expect karte hain.

Q35. Java ke 'this' aur Python ke 'self' mein difference?

Dono current object. Java mein this implicit hai (parameter mein likhna nahi padta, naming conflict pe use hota hai); Python mein self **explicit first parameter** hai.

Q36. Instance variable kya hai? Har object ke liye alag kyun hota hai?

Constructor mein self.x = ... se bane variables. **Har object ka value alag** hota hai (SBI ka PIN alag, HDFC ka alag, har customer ka balance alag) kyunki self har baar alag object hota hai. Isiliye instance variables constructor ke andar banate hain.

Q37. Method A se method B ko call karna ho to?

self.B() likhte hain. Direct B() likhoge to NameError aayega (B class ke andar defined hai, bahar ke scope mein nahi).

PART 3

Fraction: Apna Data Type + Operator Overloading

Video Ch.7: Create own data type (Fraction) - add, sub, mul, div

Python mein fraction ka built-in type nahi hai (5/6 turant decimal ban jata hai). Hum **Fraction class** banate hain jisme numerator aur denominator data hai, aur +, -, *, / ka apna logic magic methods se likhte hain. Yehi 'apna data type' banana hai.

```
class Fraction:
    def __init__(self, n, d):
        self.num = n
        self.den = d

    def __str__(self):                      # print(x) ke liye
        return '{}/{}'.format(self.num, self.den)

    def __add__(self, other):               # x + y
        n = self.num * other.den + other.num * self.den
        d = self.den * other.den
        return '{}/{}'.format(n, d)

    def __sub__(self, other):               # x - y
        n = self.num * other.den - other.num * self.den
        return '{}/{}'.format(n, self.den * other.den)

    def __mul__(self, other):               # x * y
        return '{}/{}'.format(self.num * other.num, self.den * other.den)

    def __truediv__(self, other):           # x / y  (reciprocal se multiply)
        return '{}/{}'.format(self.num * other.den, self.den * other.num)

x = Fraction(3, 4); y = Fraction(1, 2)
print(x)        # 3/4
print(x + y)    # 10/8
print(x - y)    # 2/8
```

Q38. Fraction object ko print karne par <__main__.Fraction object at 0x...> kyun dikhta hai?

Kyunki class banane wale (tumne) ne nahi bataya ki object **kaisa dikhna chahiye**. Python ko sirf itna pata hai ki ye Fraction class ka object hai aur kis memory location par hai. Solution: __str__ define karo.

Q39. x + y do Fraction objects pe kyun fail hota hai? Error kya aata hai?

TypeError: unsupported operand type(s) for +: 'Fraction' and 'Fraction'. Kyunki class ke creator ne **+ ka logic define nahi kiya** (jaise list + list ya set + set ka alag behavior hota hai). __add__ likhna padega.

Q40. Operator overloading kya hai? Python mein kaise karte hain?

Kisi operator (+, -, *, ==, <) ko apne class ke objects ke liye **custom behavior** dena. Python mein magic methods se: + -> __add__, - -> __sub__, * -> __mul__, / -> __truediv__, // -> __floordiv__, % -> __mod__, ** -> __pow__, == -> __eq__, < -> __lt__, > -> __gt__.
Jab bhi x + y likhte ho Python andar se x.__add__(y) call karta hai.

Q41. __add__(self, other) mein self aur other kaun hote hain?

Binary operator hai to 2 objects chahiye. x + y mein **self = x** (left) aur **other = y** (right). self.num = x ka numerator, other.num = y ka numerator.

Q42. Fraction division mein hum kya trick use karte hain?

Second fraction ko **ulta (reciprocal)** karke multiply: (a/b) / (c/d) = (a*d)/(b*c). Code mein: n = self.num*other.den aur d = self.den*other.num.

Q43. Fraction ko simplify (reduce) kaise karoge?

math.gcd se divide: g = gcd(n, d); n//=g; d//=g. Sahi design mein __add__ ko string nahi, **Fraction object return** karna chahiye taaki chain (x+y)*z chal sake.

```
from math import gcd
def __add__(self, o):
    n = self.num*o.den + o.num*self.den
    d = self.den*o.den
    g = gcd(n, d)
    return Fraction(n//g, d//g)   # object return
```

Q44. Python mein operator overloading ke liye 'operator' keyword hota hai? (C++ se tulna)

Nahi. C++ mein operator+ function likhte hain; Java mein operator overloading hoti hi nahi (sirf String ka + built-in). Python mein sirf magic methods.

Q45. Reverse operators (__radd__) kya hote hain?

5 + obj mein left side int hai, wo obj ko nahi jaanta. Tab Python obj.__radd__(5) try karta hai. Isse commutative behavior milta hai.

Q46. Augmented assignment (+=) ke liye kaun sa method?

__iadd__. Na likho to Python __add__ se kaam chala leta hai (x = x + y).

Q47. Real use-case: coordinate geometry / matrix class

Ek student ne **Point** aur **Line** classes banayi: distance between points, point on line check, equilateral triangle check, matrix classes... Is tarah ke custom data types koi bhi library mein daal ke dusre use kar sakte hain - yehi OOP ki power hai (aur placement mein impress karta hai).

PART 4

Encapsulation

Video Ch.8: Encapsulation + Notes Part 1

Encapsulation = Data + Methods ek unit mein wrap karna + data ko bahar se directly access hone se bachana.
Analogy: **Capsule** - andar ki dawai (data) bahar se nahi dikhti, ek cover (class) mein band hai. Access karna ho to controlled gateway (getter/setter methods) se.

4.1 Problem: Data openly accessible

```
sbi = ATM()
sbi.balance = 'abc'        # kisi ne string daal di!
sbi.deposit(5000)          # TypeError: str + int  -> existing code toot gaya
sbi.pin                    # PIN seedha dikh gaya
# Junior programmer ki ek galti poora system bigaad sakti hai
```

4.2 Solution: Hide (private) + controlled access

```
class ATM:
    def __init__(self):
        self.__pin = ''           # double underscore -> private jaisa
        self.__balance = 0

    def get_pin(self):            # getter  (sirf dekhne ke liye)
        return self.__pin

    def set_pin(self, new_pin):   # setter  (apne rules ke saath)
        if type(new_pin) == str:
            self.__pin = new_pin
            print('PIN changed')
        else:
            print('PIN string honi chahiye')

sbi = ATM()
sbi.__pin            # AttributeError (hide ho gaya)
sbi.set_pin('1234')  # sahi tareeka
```

Q48. Encapsulation kya hai? (Interview mein 3 line answer)

Data (variables) aur us par kaam karne wale code (methods) ko **ek single unit (class)** mein bundle karna, aur kuch components ka direct access **restrict** karna. Isse data safe rehta hai aur sirf class ke methods se hi modify hota hai.

Q49. Encapsulation ki zaroorat kyun padti hai?

Agar data public chhod do to koi bhi (bahar ka code ya junior programmer) galat value daal sakta hai (balance = 'abc'), jisse existing logic toot sakta hai. Data hide karke aur setter mein validation lagakar hum **apne rules enforce** kar sakte hain.

Q50. Python mein data private kaise banate hain?

Variable/method ke naam ke aage **double underscore (__)** lagao: self.__balance. Java mein private keyword hota hai.

Q51. Python mein true private hota hai? Name mangling kya hai?

Nahi. Python __balance ko internally _ClassName__balance se replace kar deta hai - isko **name mangling** kehte hain. Isliye bahar se sbi.__balance nahi chalta, par sbi._ATM__balance chal jata hai.

```
sbi = ATM()
print(sbi._ATM__pin)        # access ho gaya! (mangled name)
sbi._ATM__balance = 'hack'  # modify bhi ho jayega
print(sbi.__dict__)         # {'_ATM__pin': '', '_ATM__balance': 0}
```

Q52. Python mein private kyun nahi rakha gaya? (Philosophy)

Python ki philosophy: **'We are all consenting adults'**. Language ne chhupane ka strong feature isliye nahi diya ki maybe kabhi zaroorat pad jaye to developer use kar sake. Sirf ye signal dete hain ki 'ye internal hai, mat chhedo'. Agar koi phir bhi chhede to wo uski zimmedari.

Q53. Obj.__balance = 100 bahar se likh diya, error nahi aaya. Kyun?

Bahar se sbi.__balance = 100 likhne se class ke andar ek **naya variable** **__balance** bana (mangled nahi, kyunki class ke bahar hai). Pehle wala _ATM__balance untouched hai. Naya variable koi use nahi kar raha isliye existing logic safe hai - par ye confusion ka source hai.

Q54. Getter aur Setter kya hain? Jab data hide kar diya to ye kyun?

Getter = value padhne ka method (get_pin), **Setter** = value badalne ka method (set_pin). Data hide karke bhi hum sab kuch band nahi karte - controlled access dete hain, aur **setter ke andar apna logic** (validation) lagate hain. 'Dekhna hai to function se dekho, badalna hai to mere logic se guzar ke badlo.'

Q55. Setter use karne ka fayda jab sidha sbi.pin = x bhi kar sakte the?

Setter se **validation / rules** enforce hote hain (PIN hamesha string ho, balance negative na ho, length 4 ho). Seedha assign karne par ye checks bypass ho jate. Sath hi baad mein internal implementation badal sakte ho bina bahar ka code todhe.

Q56. Encapsulation ka formula / summary?

Encapsulation = Hide + Controlled access. Data member ko private banao, uske liye getter/setter public methods do - dono saath mein kaam karte hain (jaise capsule mein dawai powder karke ek cover mein).

Q57. Method ko bhi private bana sakte hain?

Haan - def __helper(self). Internal helper methods (jaise PIN check logic) ko bahar se hide kar sakte ho.

Q58. @property decorator kya hai? (Pythonic getter/setter)

@property se method ko attribute ki tarah access kar sakte hain (obj.balance), aur @balance.setter se validation add. Bahar se simple dikhta hai par andar control hai.

```
class Account:
    def __init__(self): self.__balance = 0

    @property
    def balance(self):            # getter
        return self.__balance

    @balance.setter
    def balance(self, v):         # setter
        if v < 0: raise ValueError('negative nahi')
        self.__balance = v

a = Account()
a.balance = 500      # setter chala
print(a.balance)     # getter chala
```

Q59. _x, __x, __x__ mein difference?

_x: protected by convention (sirf signal, enforce nahi). **__x**: name mangling (private jaisa). **__x__**: magic/dunder method (Python reserved, apne naam mat banao).

Q60. Encapsulation aur Abstraction mein difference?

Encapsulation = data hiding (data + methods bundle karke access restrict) - *HOW data protected hai*. **Abstraction** = implementation details hide karke sirf essential features dikhana (abstract class/interface) - *WHAT user ko dikhe*. Encapsulation implementation-level, abstraction design-level.

Q61. Class diagram mein private aur public kaise dikhate hain?

Private = **- (minus)**, public = **+ (plus)**. Constructor ko bhi - dikhate hain kyunki user control nahi karta.

Java access modifiers (Notes se - zaroor yaad rakho)

Modifier	Same class	Same package	Subclass	Everywhere
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Language	Private kaise	Strict?
Java	private keyword	Haan (compile error)
C++	private: section	Haan
Python	__name (name mangling), _name (convention)	Nahi - bypass ho sakta hai

PART 5

Reference Variable, Pass by Reference, Collections

Video Ch.9 - Ch.11

5.1 Reference Variable

ATM() likhne se object memory (heap) mein ek address par bana. Agar tum use kisi variable mein store nahi karte to wo kho jaata hai. Jis variable mein hum object ka address / reference store karte hain use **Reference Variable** kehte hain. sbi = ATM() mein **sbi** reference variable hai, **ATM()** object hai.

Q62. Reference variable kya hota hai?

Wo variable jisme object ka **memory address (reference)** store hota hai. Object khud variable ke andar nahi hota - variable use point karta hai. Object ko use karne ke liye reference zaroori hai, warna wo lost ho jata hai (garbage collect).

Q63. Object bana par kisi variable mein store nahi kiya, to?

ATM() akela likhne se object ban gaya aur constructor chala, par uska address kahin store nahi hua. Ab use access nahi kar sakte (lost). Python ka garbage collector use delete kar dega.

Q64. Do variables same object ko point kar sakte hain?

Haan: a = Cust('Nitish'); b = a - dono same object ko point karte hain (id(a) == id(b)). b se change karoge to a mein bhi dikhega.

5.2 Pass by Reference

```
class Customer:
    def __init__(self, name, gender):
        self.name = name
        self.gender = gender

def greet(customer):                    # class ke BAHAR ka function
    if customer.gender == 'male':
        print('Hello', customer.name)
    else:
        print('Hello Ms.', customer.name)
    customer.name = 'Ankita'            # object edit kiya!  (bahar bhi change)

c = Customer('Nitish', 'male')
greet(c)
print(c.name)          # Ankita  -> original object change ho gaya

def greet2(customer):
    new = Customer('Raj', 'male')      # nayi cheez bana ke
    return new
c2 = greet2(c)         # functions object return bhi kar sakte hain
```

Q65. Python mein objects function ko kaise pass hote hain? Pass by value ya reference?

Pass by (object) reference. Function ko object ka reference milta hai, copy nahi. Isliye function ke andar object ke attributes badlo to **bahar ka original object bhi change** hota hai (dono same object ko point karte hain).

Q66. Function ke andar customer = Customer(...) (reassign) kiya to bahar ka object badlega?

Nahi. Reassign karne se sirf function ka local reference naye object ko point karta hai. Bahar wala variable ab bhi purane object ko point karta hai. Sirf **attribute mutate** karne se bahar tak asar aata hai, variable reassign se nahi.

```
def f(cust):
    cust.name = 'A'        # bahar tak asar
    cust = Customer('B','m')  # sirf local
c = Customer('N','m')
f(c)
print(c.name)     # A
```

Q67. Kya function apni class ke object ko argument le sakta hai aur return kar sakta hai?

Haan. Python mein sab kuch object hai, to int, str, list ki tarah apni class ka object bhi pass aur return kar sakte hain.

5.3 Mutable vs Immutable (id ke saath)

Mutable (change in-place)	Immutable (change = naya object)
list, dict, set, user-defined objects, bytearray	int, float, str, tuple, frozenset, bool, bytes

```
L = [1, 2, 3]
print(id(L))
L.append(4)
print(id(L))        # SAME id -> mutable

def change(l):
    l.append(100)    # original list badal gayi!
change(L)            # L = [1,2,3,4,100]

def safe(l):
    l = l[:]         # clone, original safe     (ya l.copy())
    l.append(100)

T = (1, 2, 3)
def f2(t):
    t = t + (4,)     # nayi tuple, original same
f2(T); print(T)      # (1, 2, 3)
```

Q68. Python mein objects mutable hote hain ya immutable? Kaise pata kare?

User-defined class ke objects **mutable** hote hain. Check: attribute edit karne se pehle aur baad id(obj) print karo - same aata hai to mutable (in-place change ho raha hai). Agar id badal jaye to immutable.

Q69. Function mein list bhejne par original list kyun change ho jaati hai? Bachne ka tareeka?

List mutable hai aur reference se jaati hai, isliye append original ko hi change karta hai. Bachne ke liye pass karne se pehle **clone** banao: l[:], list(l), l.copy() (nested ho to copy.deepcopy) ya immutable **tuple** bhejo.

Q70. Tuple pe append() ya item change kyon nahi chalta?

Tuple immutable hai. Naya data chahiye to t = t + (5,) likho - ye naya tuple banata hai (naya id), original same.

Q71. Function ko mutable object bhejne ka risk batao.

Function ke andar hone wale operations se original object me **permanent change** ho sakta hai. Isliye mutable bhejte time sochke (ya copy karke) bhejo.

5.4 Collection of Objects

```
class Customer:
    def __init__(self, name, gender):
        self.name, self.gender = name, gender
    def intro(self):
        print('I am', self.name, 'and I am', self.gender)

c1 = Customer('Nitish', 'male')
c2 = Customer('Ankita', 'female')
c3 = Customer('Raj', 'male')

L = [c1, c2, c3]           # list of objects (dict/tuple me bhi)
for i in L:
    i.intro()              # har object pe method chalta hai
print(L)                   # [<Customer object at 0x..>, ...]  (__repr__ chahiye)
```

Q72. List/Dict/Tuple mein objects store ho sakte hain?

Haan, objects bhi Python ki baaki cheezon jaise hain. List me store karke loop chalao aur har object pe method call karo. Dict ki value, tuple ke elements - sab me chal sakta hai. **Set me** daalne ke liye object **hashable** hona chahiye (__hash__ aur __eq__).

Q73. Objects ki list print karne par addresses kyun dikhte hain? Kaise sahi karein?

Class mein __repr__ (ya __str__) define nahi hai. List print karte waqt har element ka __repr__ use hota hai - isliye __repr__ likho.

Q74. Objects ki list ko sort kaise karoge (jaise age se)?

sorted(L, key=lambda c: c.age) ya class mein __lt__ define karke sorted(L). Descending: reverse=True.

PART 6

Static Variables and Methods

Video Ch.12

Do tarah ke variables: **Instance variable** (har object ka alag - constructor mein self.x) aur **Static / Class variable** (poori class ke liye ek hi, saare objects mein shared - **constructor ke bahar** class ke andar).
Use: bank ka naam, college ka naam, IFSC code, **serial number counter**.

```
class ATM:
    __counter = 1                  # STATIC: constructor ke bahar, class level

    def __init__(self):
        self.pin = ''              # instance variable
        self.__balance = 0
        self.cid = ATM.__counter   # class name se access
        ATM.__counter += 1         # sabke liye ek hi counter badha

    @staticmethod                  # self nahi chahiye
    def get_counter():
        return ATM.__counter

    @staticmethod
    def set_counter(new):
        if type(new) == int: ATM.__counter = new
        else: print('Not allowed')

c1, c2, c3 = ATM(), ATM(), ATM()
print(c1.cid, c2.cid, c3.cid)      # 1 2 3
print(ATM.get_counter())           # 4  (bina object ke call)
ATM.__counter                      # AttributeError (private)
```

Q75. Static (class) variable aur instance variable mein difference?

Instance: har object ke liye alag copy, constructor ke andar self.x se bante hain. **Static/Class:** class ke liye ek hi copy, saare objects share karte hain, constructor ke **bahar** define hote hain, access ClassName.var se. Memory mein bhi ek hi baar.

Q76. Static variable ki zaroorat kab padti hai?

Jab saare objects ke beech **shared value** chahiye: bank/college ka naam, IFSC code, **auto serial number** (har naye object ko 1, 2, 3...). Instance variable se counter nahi ban sakta kyunki har object ke liye wo reset ho jata.

Q77. Counter instance variable se kyun nahi bana sakte?

Constructor har naye object pe chalta hai aur har object ke liye nayi self.counter = 0 banta hai - to har object ka counter 1 hi rahega. Shared counter class level pe chahiye.

Q78. Static variable ko access karne ka sahi tareeka? obj.counter likh sakte hain?

ClassName.var (ATM.counter). obj.counter padhna chal jata hai, par **obj.counter = 10 likhne par** class variable nahi badalta - object mein ek **naya instance variable** ban jata hai jo class wale ko shadow karta hai. Isliye hamesha class name se.

```
ATM.counter = 5
c1 = ATM()
c1.counter = 99     # sirf c1 ka naya instance var
print(ATM.counter)  # 5  (unchanged)
```

Q79. Static method kya hota hai? Kab use karte hain?

Wo method jo **static variables ke saath kaam** karta hai aur jise **object ki zaroorat nahi** (self nahi chahiye). @staticmethod decorator se. Call: ClassName.method(). Instance data ko touch nahi karta.

Q80. Static method me self kyun nahi lete?

Kyunki self object ka reference hota hai. Static method object se bandha nahi hota, class-level cheez par kaam karta hai. Self lene ka koi matlab nahi.

Q81. Static variable private ho to bahar se kaise access/change karoge?

Static getter/setter methods se: ATM.get_counter(), ATM.set_counter(5). Setter mein type-check jaisa logic daal sakte ho.

Q82. @staticmethod vs @classmethod vs instance method?

Table dekho neeche.

Type	Pehla param	Kya access kar sakta hai	Use
Instance method	self	Object + class data	Normal behavior
@classmethod	cls (class khud)	Class variables; subclass-aware	Alternate constructors, factory
@staticmethod	kuch nahi	Kuch nahi automatic (utility)	Helper jo class se related ho

Q83. Java mein static keyword ke rules?

Static method me this use nahi hota, sirf static members access kar sakta hai. Static members class load hote hi bante hain, object ke bina access hote hain (ClassName.member). Static block class load ke time chalta hai.

Q84. Static vs Instance members (Notes)?

Static: class level, sabhi objects mein shared, object ke bina access. **Instance:** har object ki apni copy. Static method mein this/self nahi.

PART 7

Class Relationships: Aggregation (Has-A)

Video Ch.13

Bade application mein ek nahi, **multiple classes** hoti hain (Product, Customer, Address, Review). Unke beech 2 major relationships: **Aggregation (Has-A)** aur **Inheritance (Is-A)**.
Aggregation: ek class ka object doosri class ke object ko *contain* karta hai (Customer HAS-A Address). Customer khud address ka kaam nahi karta, Address class se karwata hai.

```
class Customer:
    def __init__(self, name, gender, address):    # address ek OBJECT hai
        self.name = name
        self.gender = gender
        self.address = address

    def edit_profile(self, new_name=None, new_city=None,
                     new_pin=None, new_state=None):
        if new_name:
            self.name = new_name
        self.address.edit_address(new_city, new_pin, new_state)  # kaam Address ko

class Address:
    def __init__(self, city, pincode, state):
        self.city, self.pincode, self.state = city, pincode, state

    def edit_address(self, city, pin, state):
        self.city = city or self.city
        self.pincode = pin or self.pincode
        self.state = state or self.state

add1 = Address('Kolkata', 700001, 'WB')
cust = Customer('Nitish', 'male', add1)    # aggregation
print(cust.address.city)                   # Kolkata
cust.edit_profile('Ankit', 'Gurgaon', 122011, 'Haryana')
```

Q85. Aggregation kya hai? Example do.

Do classes ke beech **Has-A** relationship: ek class ka object doosri class ke object ko attribute ki tarah rakhta hai. Example: Customer has Address, Department has Professors, Car has Engine. Customer apna address-related kaam khud nahi karta, Address ke methods ko call karta hai.

Q86. Aggregation ko code mein kaise implement karte hain?

Constructor mein doosri class ka **object** parameter ki tarah lo aur self.address = address store karo. Bahar pehle Address ka object banao, fir Customer ko pass karo.

Q87. Aggregation mein class diagram kaise dikhta hai?

Container class (Customer) ki taraf **diamond**, contained class (Address) ki taraf line. Diamond jis taraf = 'whole'.

Q88. Association vs Aggregation vs Composition? (Must-know)

Association: general 'uses-a', independent lifecycles (Teacher - Student). **Aggregation:** weak ownership 'has-a' - container destroy hone par contained zinda rehte hain (Department has Professors). **Composition:** strong ownership - container destroy to contained bhi destroy (House has Rooms).

Q89. Customer ka edit_profile ek hi class mein kyun nahi likha? Address ko alag kyun rakha?

Single Responsibility. Address ka kaam Address class ko hi karna chahiye - city/pincode ka logic ek jagah rahe. Customer sirf delegate karta hai. Kal address change ho to sirf Address class touch hogi.

Q90. Default arguments (new_name=None) edit_profile mein kyun?

Taaki user sirf wahi cheez bheje jo badalni hai. Na bheje to purani value rahe (if new_name: check).

PART 8

Inheritance, super() aur MRO

Video Ch.14 - Ch.19 + Notes

8.1 Inheritance basics

Real life: papa ki property beta ko mili (par beta ki kamai papa ko nahi) - direction hamesha **upar se neeche**. Biological: genes se features aate hain. Software mein: **Parent (Base/Super) class** ki cheezein **Child (Derived/Sub) class** inherit karti hai.
Problem jo solve hota hai: Udemy jaisi site mein Student aur Instructor dono ko login/register chahiye - code do baar likhoge to **DRY (Don't Repeat Yourself)** toot jata hai. Solution: **User** class banao (login, register) aur Student, Instructor dono ko User se inherit karwao.

```
class User:                     # Parent / Base / Super class
    def login(self):    print('login')
    def register(self): print('register')

class Student(User):            # Child / Derived / Sub class
    def enroll(self): print('enroll in course')
    def review(self): print('review')

s = Student()
s.enroll()        # apna
s.login()         # User se inherit kiya
u = User()
u.enroll()        # AttributeError: 'User' object has no attribute 'enroll'
```

Q91. Inheritance kya hai?

Ek class (child/derived) ka existing class (parent/base) ki properties aur behaviors ko **adopt/reuse** karna. Syntax Python: class Child(Parent):

Q92. Inheritance ka sabse bada fayda?

Code reusability (DRY). Common code parent mein ek baar, child classes inherit karti hain. Bade applications mein code size chhota, optimized aur maintainable rehta hai.

Q93. Child class parent se kya-kya inherit karti hai?

3 cheezein: 1) saare **data members** (variables), 2) saare **methods** (member functions), 3) **constructor**. **Private members** inherit nahi hote (access nahi milta).

Q94. Inheritance ki direction kya hoti hai? Parent child ke methods access kar sakta hai?

Hamesha upar ki taraf (child parent se le sakta hai). Parent child ke methods/attributes access nahi kar sakta - User().enroll() -> AttributeError. Jaise papa ki property beta ki hai, par beta ki kamai papa ki nahi.

Q95. Child class ka object parent ke methods call kar sakta hai?

Haan, saare public methods. Private ya name-mangled attributes direct nahi.

Q96. Inheritance mein Is-A relationship ka kya matlab?

Student IS-A User, Dog IS-A Animal. Agar 'is-a' sach nahi hai (Car is-a Engine galat hai) to inheritance mat use karo - **composition** (has-a) use karo.

Q97. Python mein parent class ko kya bolte hain?

Parent = Base class = Super class. Child = Derived class = Sub class. Interview mein dono naam aate hain.

8.2 Constructor aur private ka inheritance

```
class Phone:
    def __init__(self, price, brand, camera):
        print('Inside Phone constructor')
        self.__price = price          # private
        self.brand = brand
        self.camera = camera
    def buy(self): print('buying phone')

class SmartPhone(Phone):              # koi constructor nahi
    pass

s = SmartPhone(20000, 'Apple', 13)    # Phone ka __init__ chal gaya
print(s.brand)                        # Apple
print(s.__price)                      # AttributeError (private inherit nahi)
```

Q98. Child ke paas constructor nahi hai aur parent ke paas hai - object banane par kya hoga?

Parent ka constructor automatically chalta hai. Python child class me __init__ khojta hai, nahi milta to parent me jaata hai.

Q99. Child ke paas bhi constructor ho to parent ka chalega?

Nahi. Child ka constructor hi chalega, parent ka automatically nahi. Parent ke variables bane hi nahi to baad me unhe access karne par AttributeError. Parent ka constructor **super().__init__()** se explicitly call karna padta hai.

```
class Parent:
    def __init__(self, n): self.num = n
class Child(Parent):
    def __init__(self, n):
        pass                  # parent init nahi chala
c = Child(5)
print(c.num)                  # AttributeError: no attribute 'num'
```

Q100. Private members inherit hote hain?

Child object parent ke **private (__x) members access nahi kar sakta**. Python mein __x mangled ho kar _Parent__x ban jata hai, isliye child ke andar self.__x ka matlab _Child__x hota hai (alag naam). Child ko chahiye to parent ke public getter use karo ya variable ko **protected (_x)** rakho.

Q101. Child class ke object ko parent ka protected (_x) mil sakta hai?

Haan. Single underscore sirf convention hai, access ho jata hai. Java mein protected subclass aur same package me access deta hai.

8.3 super() keyword

```
class Phone:
    def __init__(self, price, brand, camera):
        self.__price, self.brand, self.camera = price, brand, camera
    def buy(self): print('Phone.buy')

class SmartPhone(Phone):
    def __init__(self, price, brand, camera, os, ram):
        super().__init__(price, brand, camera)   # parent ka common kaam parent se
        self.os = os                              # sirf apna specific kaam
        self.ram = ram
    def buy(self):
        print('SmartPhone.buy')
        super().buy()                             # parent ka method bhi chalao

s = SmartPhone(20000, 'Samsung', 12, 'Android', 4)
s.buy()    # SmartPhone.buy  ->  Phone.buy

# super() class ke BAHAR kaam nahi karta:
# s.super().buy()   -> AttributeError
```

Q102. super() kya hai? Kab use karte hain?

Parent class ke **methods aur constructor** ko child ke andar se call karne ka tareeka. Jo kaam parent karta hai (common attributes set karna) wo parent se karwao, aur sirf child-specific cheezein child mein likho - phir se likhna nahi padta.

Q103. super() se kya access ho sakta hai aur kya nahi?

Ho sakta hai: parent ke methods aur constructor. **Nahi:** parent ke variables (super().brand nahi chalega; self.brand chalega). Aur **class ke bahar** super() use nahi kar sakte - AttributeError.

Q104. super().__init__() ko pehle hi likhna zaroori kyun hai?

Convention/best practice: parent ke attributes pehle set ho jayein, taaki child unhe use kar sake aur child ki value parent ko accidentally overwrite na kar de. **Java mein super(...) constructor ki pehli line hi honi chahiye (compile rule)**; Python mein force nahi, par pehle likhna sahi practice hai.

Q105. super() aur self.__class__.__init__ mein kya fark? Direct Parent.__init__(self) kyun nahi?

Parent.__init__(self, ...) bhi chalta hai par parent ka naam hardcode ho jata hai aur multiple inheritance (MRO) me problem. **super()** MRO follow karta hai, isliye hamesha preferred.

Q106. Java me super aur this kya hain?

this = current object ka reference (naming conflict solve, constructor chaining this(...)). **super** = parent class ka reference (parent method/constructor). Dono constructor ki pehli line me hi aate hain.

Q107. Predict: parent ke constructor me self.num = 100 hai, child me self.num = 10 set karke show() me print(super().num) likha to?

Error aayega. super() sirf methods/constructor deta hai, variable nahi. Aur self.num to child wale 10 ko hi dikhata (kyunki same object pe overwrite hua). Parent ki value chahiye to pehle store karo ya parent ka method call karo.

Q108. Predict: parent __init__ me self.num=100, child __init__ me super().__init__() ke baad self.num=10; show() me print(self.num) ka output?

10. self ek hi object hai (child ka), parent ne 100 set kiya fir child ne usi variable ko 10 kar diya. Order matter karta hai.

8.4 Types of Inheritance

Type	Structure	Python	Java	C++
Single	A -> B	Yes	Yes	Yes
Multilevel	A -> B -> C (dada-papa-bachha)	Yes	Yes	Yes
Hierarchical	A -> B, A -> C (ek parent, kai child)	Yes	Yes	Yes
Multiple	B, C -> D (do parents)	Yes	Nahi (interfaces se)	Yes
Hybrid	Upar ke kisi combination ka mix	Yes	Partial	Yes

```
class A: pass
class B(A): pass          # single
class C(B): pass          # multilevel: C -> B -> A
class D(A): pass          # hierarchical: A ke do child (B, D)
class E(B, D): pass       # multiple inheritance (do parent)
```

Q109. Inheritance ke types batao, har ek ka example.

Single: Smartphone(Phone). **Multilevel:** Smartphone -> Phone -> Product (child sabke methods access kar sakta hai). **Hierarchical:** ek parent (Phone) aur kai child (Smartphone, FeaturePhone). **Multiple:** ek child, do parents (Smartphone(Phone, Product)). **Hybrid:** in sabka combination.

Q110. Java mein multiple inheritance kyun nahi hota? Python mein hota hai?

Java designers ne **ambiguity (diamond problem)** se bachne ke liye classes ka multiple inheritance disallow kiya; interfaces se type-multiple-inheritance milta hai. Python me allowed hai aur **MRO (C3 linearization)** se resolve karta hai.

Q111. Multilevel me child ko grandparent ke methods mil sakte hain?

Haan. Child apne parent ke saath saath grandparent ke members bhi inherit karta hai (jab tak private na ho).

Q112. Hybrid inheritance kya hai?

Single, multilevel, hierarchical aur multiple me se kisi bhi **combination**. Diagram me kahin multilevel, kahin hierarchical aur kahin multiple dikhta hai.

8.5 MRO - Method Resolution Order

```
class Phone:
    def buy(self): print('buying phone')
class Product:
    def buy(self): print('buying product')
    def review(self): print('product review')

class SmartPhone(Phone, Product):      # pehle Phone, phir Product
    pass

s = SmartPhone()
s.buy()                 # buying phone  (Phone pehle likha hai)
s.review()              # product review (Phone me nahi, Product me mila)
print(SmartPhone.__mro__)
# (SmartPhone, Phone, Product, object)

# Order badal do:  class SmartPhone(Product, Phone)  ->  'buying product'
```

Q113. MRO kya hota hai? Kaise decide hota hai?

Method Resolution Order = Python kis order me classes me method khojta hai. Rule: **jis order me tumne inherit kiya (left to right)** wahi priority. Child -> Parent1 -> Parent2 -> ... -> object. Python 3 me algorithm **C3 linearization**. Check: Class.__mro__ ya Class.mro().

Q114. Dono parents me same naam ka method hai to konsa chalega?

Jo class pehle likhi hai (left-most). class S(Phone, Product) me Phone ka. Phone ka na ho to Product ka. Constructor ke liye bhi yehi rule.

Q115. Multiple inheritance me constructor kiska chalega?

MRO me jo pehle mile - child ka na ho to left-most parent ka __init__. Dusre parent ka tabhi chalega jab super() chain call kare.

Q116. Diamond Problem kya hai? Python, Java aur C++ kaise solve karte hain?

B aur C dono A se inherit karein, D dono B aur C se. A ka method dono me override ho to D ko ambiguity ki kaunsa version le. **Python:** MRO (C3) ek fixed order deta hai. **Java:** classes ka multiple inheritance disallow; interfaces ke default methods me implementing class ko explicitly override karna padta hai. **C++:** virtual inheritance se A ki ek hi copy.

```
class A:
    def hi(self): print('A')
class B(A):
    def hi(self): print('B')
class C(A):
    def hi(self): print('C')
class D(B, C): pass

D().hi()            # B
print(D.__mro__)    # D, B, C, A, object
```

Q117. Predict + explain: class A m1 -> 10; B(A) m1 me self.m1() call? Kya hoga?

Agar B.m1 ke andar self.m1() likhoge aur object C ka hai to self C ka object hai -> phir wahi method (apne aap ko) call -> **infinite recursion -> RecursionError: maximum recursion depth exceeded**. **Rule:** self ke saath jo kaam bahar object karta, wahi class ke andar hota hai. Parent ka method chahiye to super().m1() likho.

```
class A:
    def m1(self): return 20
class B(A):
    def m1(self): return self.m1() + 10   # khud ko call!
class C(B): pass
C().m1()          # RecursionError

# Fix:
class B(A):
    def m1(self): return super().m1() + 10   # 30
```

Q118. super() aur MRO kaise related hain?

super() 'parent' nahi, **MRO me agli class** ko point karta hai. Multiple inheritance me ye cooperative chain banata hai: har class ka __init__ super() se agle ko call kare to sabka chal jata hai.

Q119. Python me issubclass() aur isinstance()?

isinstance(obj, Cls): obj class ka (ya uski child class ka) hai? issubclass(Child, Parent): Child parent se inherit karta hai? Dono polymorphism ke type-check me kaam aate hain.

Q120. Composition vs Inheritance - kab kaunsa? (Notes)

Inheritance (IS-A): tight coupling, parent change = child toot sakta hai. **Composition (HAS-A):** flexible, runtime pe behavior badal sakta hai, loose coupling. Rule: **Favor composition over inheritance.** Eg: Car has an Engine (Car extends Engine galat). Inheritance tab jab sach me IS-A ho aur Liskov follow ho.

Q121. Inheritance ke nuksaan / disadvantages?

Tight coupling (parent badla to child affect), deep hierarchy confusing, fragile base class problem, multiple inheritance me diamond ambiguity, unnecessary members inherit ho jate hain.

PART 9

Polymorphism

Video Ch.16, Ch.20 + Notes

Poly = many, Morph = forms. Ek hi entity (method / operator / object) ka alag situation me alag behavior. 3 types: **Method Overriding, Method Overloading, Operator Overloading**.
Real life analogy: Family business hai (dada, papa, bhaiya wahi business karte hain). Tum software developer bane - ab tumhara career alag hai. Conflict me 'main apna hi follow karunga' = overriding.

	Compile-time (Static / Overloading)	Runtime (Dynamic / Overriding)
Kahan	Same class	Parent-child classes
Signature	Same naam, alag parameter list	Bilkul same signature
Resolve	Compile time (early binding)	Runtime (late binding, vtable)
Python me	Natively nahi (default args / *args)	Haan, naturally

9.1 Method Overriding

```
class Phone:
    def buy(self): print('buying phone')

class SmartPhone(Phone):
    def buy(self): print('buying smartphone')   # same naam -> override

SmartPhone().buy()      # buying smartphone  (child ka)
Phone().buy()           # buying phone
```

Q122. Method overriding kya hai?

Parent aur child dono me **same naam aur same signature** ka method ho aur child ka object call kare to **child wala (apna) method chalta hai**. Runtime pe decide hota hai.

Q123. Method overriding me parent ka method bhi chalana ho to?

Child ke method ke andar super().method() call karo.

Q124. Python me method hiding (static method) aur overriding me fark?

Static method ko child me same signature se define karna **hiding** hai, overriding nahi. Resolution **reference type** se compile-time pe hota hai (Java me), object type se nahi.

Q125. Overriding ke rules (Java)?

Same name + same parameters; return type same ya covariant; access modifier same ya zyada open; static/private/final override nahi hote; checked exception parent se broad nahi. @Override annotation use karo.

9.2 Method Overloading

```
# Java me chalta (alag parameters)   |  Python me ye NAHI chalega:
class Geometry:
    def area(self, radius):
        return 3.14 * radius * radius
    def area(self, l, b):                  # upar wale ko OVERRIDE kar diya
        return l * b

Geometry().area(5)       # TypeError: area() missing 1 required positional argument

# Jugaad (default argument se overloading jaisa behavior):
class Geometry:
    def area(self, a, b=0):
        if b == 0: return 3.14 * a * a
        else:      return a * b
g = Geometry()
print(g.area(4))         # circle
print(g.area(4, 5))      # rectangle  -> 20
```

Q126. Method overloading kya hai? Python me support hai?

Ek hi naam ke method ko **alag-alag inputs** dene par alag behavior. Java/C++ me natively hota hai (signature se). **Python me technically nahi hota** kyunki same naam ka doosra method pehle ko overwrite kar deta hai. Par **default arguments, *args, **kwargs, ya @singledispatch** se overloading jaisa behavior mil jata hai.

Q127. Python me method overloading ke 3 workarounds?

1) Default args: def area(self, a, b=0) **2) *args:** def area(self, *args): if len(args)==1... **3) functools.singledispatchmethod** (type ke basis pe). Ya type check: isinstance(a, int).

Q128. Overloading vs Overriding - 5 differences (Notes table)

Where: same class vs parent-child. **Signature:** alag param list vs identical. **Resolve:** compile-time vs runtime. **Binding:** early vs late. **Return type:** overloading me sirf return type alag nahi chalega; overriding me same/covariant.

9.3 Operator Overloading

Q129. Operator overloading kya hai? Example?

Kisi operator ko custom class ke liye alag behavior dena. + numbers me math add, strings me concatenate, lists me merge, **Fraction me fraction-add**. Python me magic method __add__ se. (Detail Part 3 me.)

Q130. '+' operator alag-alag types pe alag kyun kaam karta hai?

Har class ne apna __add__ define kiya hai. int ka math add, str ka concat, list ka merge. Ye **operator overloading** ka hi example hai (built-in types me).

9.4 Duck Typing (Python ki polymorphism)

```
class Duck:
    def speak(self): print('Quack')
class Dog:
    def speak(self): print('Bhow')
class Robot:
    def speak(self): print('Beep')

def make_it_speak(thing):       # type ki parwah nahi, bas speak() hona chahiye
    thing.speak()

for t in (Duck(), Dog(), Robot()):
    make_it_speak(t)
```

Q131. Duck typing kya hai?

'Agar wo duck jaise chalta aur quack karta hai to wo duck hai.' Python me object ka **type nahi, uska behavior (methods)** matter karta hai. Inheritance ke bina bhi polymorphism: koi bhi object jisme speak() ho chalega.

Q132. Polymorphism ka real-life example do.

Payment: pay() method - UPI, Card, NetBanking sab alag tareeke se implement karte hain. Shape: area() - Circle, Rectangle, Triangle alag formula. Chess: har piece ka canMove() alag.

Q133. Compile-time vs Runtime polymorphism? Runtime kaise resolve hota hai?

Compile-time: overloading/operators - compiler decide karta hai. Runtime: overriding - actual object ke type ke basis pe **virtual method table (vtable)** se. Parent reference child object ko point kare to child ka method chalega (dynamic dispatch).

Q134. Early binding vs Late binding?

Early (static): compile-time pe - overloading, static/private/final methods. **Late (dynamic):** runtime pe - overriding, virtual functions (vtable).

PART 10

Abstraction and Interfaces

Notes Part 1, 5, 6 - Java/C++/Python

```
from abc import ABC, abstractmethod

class Shape(ABC):                 # abstract class
    @abstractmethod
    def area(self): pass          # sirf declaration, body nahi

    def describe(self):           # concrete method bhi ho sakta hai
        print('Area =', self.area())

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r ** 2      # implement karna zaroori

# Shape()            -> TypeError: Can't instantiate abstract class
Circle(5).describe()
```

Q135. Abstraction kya hai?

Complex implementation details **hide** karke sirf **essential functionality** dikhana. Car chalate ho (steering, brake) par engine ka internal kaam nahi jaante. Achieve: **Abstract classes aur Interfaces** se.

Q136. Abstract class kya hai? Object bana sakte hain?

Wo class jisme ek ya zyada **abstract methods** (sirf declaration, no body) hote hain. **Direct object nahi ban sakta.** Child ko saare abstract methods implement karne padte hain, warna wo bhi abstract rahegi. Python me abc.ABC + @abstractmethod.

Q137. Abstract class vs Interface - kab kaunsa? (Most asked)

Abstract class: closely related objects ke beech shared state (instance variables) aur default behavior (IS-A).
Interface: unrelated classes me common behavior (CAN-DO), ya multiple inheritance of type chahiye. Example: Animal (abstract class) vs Flyable/Swimmable (interfaces).

Point	Abstract Class	Interface
Relation	IS-A (related classes)	CAN-DO (unrelated classes)
State	Instance variables ho sakte hain	Sirf constants (public static final)
Methods	Abstract + concrete dono	Abstract; default/static (Java 8+)
Constructor	Ho sakta hai	Nahi
Inheritance	Single extend	Multiple implement
Python me	ABC class	ABC jisme saare methods abstract (ya Protocol)

Q138. Python me interface kaise banate hain?

Python me 'interface' keyword nahi hai. **ABC class** jisme saare methods @abstractmethod ho, ya typing.Protocol (structural typing). Duck typing ki wajah se aksar zaroorat nahi padti.

Q139. Abstract class me constructor ho sakta hai?

Haan - child class ka constructor super().__init__() se call kar sakta hai. Interface me constructor nahi hota (Java).

Q140. Abstract method ko private/static/final bana sakte hain? (Java)

Nahi - abstract ka matlab child ko override karna hi hai; private/static/final override nahi hote isliye contradiction.

Q141. Abstraction aur Encapsulation me confusion kaise door karein?

Abstraction: **'kya karta hai'** dikhao, **'kaise'** chhupao (design level, interface/abstract). Encapsulation: data ko **private karke access control** (implementation level). Dono milkar kaam karte hain.

PART 11

Advanced and Tricky Questions

Notes Part 2, 6 + extra deep dive

11.1 Copy: Shallow vs Deep

```
import copy
a = [[1, 2], [3, 4]]
s = copy.copy(a)        # shallow  (a.copy() / a[:] bhi shallow)
d = copy.deepcopy(a)    # deep

a[0][0] = 99
print(s)    # [[99, 2], [3, 4]]  -> nested list share hui
print(d)    # [[1, 2], [3, 4]]   -> bilkul independent
```

Q142. Shallow copy vs Deep copy?

Shallow: sirf top-level fields copy; nested objects ke **references share** - ek me change = dusre me bhi dikhta hai. **Deep:** nested objects ki bhi nayi copy, dono independent. Java: clone() default shallow; deep ke liye copy constructor/serialization. C++: default copy ctor shallow; pointers ho to custom deep copy.

Q143. Python me b = a se copy hoti hai?

Nahi. Ye sirf naya reference hai (same object). Copy ke liye copy.copy / deepcopy ya object ka copy constructor.

11.2 Constructor tricky Qs

Q144. Kya constructor private ho sakta hai? Kab use hota hai?

Haan. **1) Singleton** (sirf ek object), **2) Utility/Helper classes** (sirf static methods, instantiation roka), **3) Factory methods** (object creation static method se control).

Q145. __new__ aur __init__ me fark?

__new__ object *banata* hai (memory allocate, class method jaisa, **pehle** chalta hai aur instance return karta hai). **__init__** us bane hue object ko *initialize* karta hai (values set). Singleton/immutable types me __new__ override karte hain.

```
class Singleton:
    _obj = None
    def __new__(cls, *a, **k):
        if cls._obj is None:
            cls._obj = super().__new__(cls)
        return cls._obj

a = Singleton(); b = Singleton()
print(a is b)     # True
```

Q146. Destructor / garbage collection kaise kaam karta hai?

Python: reference counting + cyclic GC; __del__ destructor (rarely use). **Java:** GC automatic; finalize() deprecated, resources ke liye try-with-resources. **C++:** destructor delete hote hi chalta hai (RAII).

Q147. Virtual destructor kya hai aur zaroori kyun? (C++)

Base pointer se derived object delete karo aur base destructor non-virtual hai to sirf base ka destructor chalta hai -> derived resources ka **memory leak**. Base destructor virtual karne par pehle derived, phir base destructor chalta hai.

Q148. Object slicing kya hai? (C++)

Derived object ko **by value** base class object me assign karne par derived-specific members aur vtable pointer **slice off** ho jate hain, sirf base portion bachta hai. Fix: pointers ya references use karo.

Q149. Virtual function aur vtable?

Virtual function runtime pe sahi override choose karwata hai. Har class ka **vtable** (function pointers ki table) hota hai aur har object me **vptr** uski taraf point karta hai.

11.3 Python specific deep questions

Q150. __slots__ kya hai?

Class me __slots__ = ('a','b') likhne se __**dict**__ **nahi banta** - memory kam, attribute access thoda fast, aur naye attributes add nahi hote.

Q151. @dataclass kya hai?

Boilerplate kam karne wala decorator: __init__, __repr__, __eq__ auto-generate. @dataclass class P: x:int; y:int. Immutable ke liye frozen=True.

Q152. __eq__ define karo to hash ka kya hoga?

Python __hash__ ko None kar deta hai (unhashable) jab tak tum explicitly __hash__ na likho. Isliye set/dict key banana hai to dono define karo.

Q153. Object ka dictionary form kaise dekhein?

obj.__dict__ - saare instance attributes dict me. Name mangling yahin dikhti hai: {'_ATM__pin': ''}.

Q154. is aur == me fark?

== value/equality (__eq__), is identity (same object? id same). a = [1]; b = [1]; a == b -> True; a is b -> False.

Q155. Python me interfaces/abstract classes ke bina polymorphism?

Duck typing se. Object ka type nahi, methods matter karte hain.

Q156. Metaclass kya hota hai?

Class ki class. Python me class bhi ek object hai jo type ka instance hai. Metaclass classes ke creation ko customize karta hai (ORMs, Django models).

Q157. Mixin kya hota hai?

Chhoti class jo koi specific behavior (logging, serialization) deti hai aur multiple inheritance se mix ki jati hai. Akeli instantiate nahi ki jati.

Q158. Immutable class kaise banate hain? (Notes)

Java: 1) class final, 2) saare fields private final (constructor me assign), 3) no setters, 4) mutable fields (Date, list) ki constructor aur getters me **deep clone**. **Python:** @dataclass(frozen=True), ya __setattr__ block, ya namedtuple.

Q159. final keyword (Java) / const (C++)?

Variable pe: value change nahi hoti. Method pe: override nahi ho sakta. Class pe: extend nahi ho sakti (jaise String). C++ me const variable/method pe; final C++11 se.

Q160. Java me finalize/equals/hashCode contract?

Do equal objects ka hashCode same hona chahiye. equals override karo to hashCode bhi override karo, warna HashMap/HashSet galat behave karenge.

PART 12

SOLID Principles + Design Patterns

Notes Part 7, 8

12.1 SOLID

Letter	Principle	Meaning	Example
S	Single Responsibility	Class ka ek hi kaam / reason to change	Invoice calculation aur printing alag classes
O	Open/Closed	Extension ke liye open, modification ke liye closed	Naya Shape add karo, purana code mat chhedo
L	Liskov Substitution	Child ko parent ki jagah use karo to program na toote	Penguin extends Bird(fly) - violation
I	Interface Segregation	Bade interface ki jagah chhote specific interfaces	Printer, Scanner alag interfaces
D	Dependency Inversion	High-level modules abstraction pe depend karein, concrete pe nahi	Service ko interface inject karo (DI)

```
# D - Dependency Inversion
class Payment(ABC):
    @abstractmethod
    def pay(self, amt): pass
class UPI(Payment):
    def pay(self, amt): print('UPI', amt)
class Card(Payment):
    def pay(self, amt): print('Card', amt)

class Order:
    def __init__(self, payment: Payment):   # abstraction pe depend
        self.payment = payment
    def checkout(self, amt): self.payment.pay(amt)

Order(UPI()).checkout(500)    # kal Card() bhi bina Order badle
```

Q161. SOLID ke 5 principles ek-ek example ke saath?

Upar table dekho. Yaad rakhne ki trick: **S**ingle job, **O**pen to extend, **L**iskov (child parent jaisa), **I**nterface chhote, **D**epend on abstraction.

Q162. Liskov Substitution violation example?

Bird class me fly() hai, Penguin extends Bird. Penguin.fly() exception/no-op karega - parent ki jagah child daalne se program toot gaya. Fix: Bird -> FlyingBird / NonFlyingBird alag.

Q163. Open/Closed principle kaise follow karte hain?

Polymorphism/abstraction se. Naya Shape banao jo Shape abstract class implement kare; area calculator code (loop with shape.area()) change nahi karna padta.

12.2 Design Patterns

```
# Singleton
class Logger:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

# Factory
class ShapeFactory:
    @staticmethod
    def create(kind):
        if kind == 'circle': return Circle()
        if kind == 'square': return Square()
        raise ValueError('unknown')

# Observer
class Channel:
    def __init__(self): self.subs = []
    def subscribe(self, s): self.subs.append(s)
    def upload(self, v):
        for s in self.subs: s.notify(v)   # sabko notification

# Strategy
class HourlyFee:
    def fee(self, hrs): return hrs * 20
class FlatFee:
    def fee(self, hrs): return 100
class Parking:
    def __init__(self, strategy): self.strategy = strategy
    def bill(self, hrs): return self.strategy.fee(hrs)
```

Q164. Singleton pattern kya hai?

Class ka sirf ek instance. Private constructor + static getInstance() (Java) / __new__ override (Python). Thread-safety ke liye double-checked locking ya enum singleton. Use: Logger, Config, DB connection pool.

Q165. Factory pattern kya hai?

Object creation ka logic ek factory method me; client ko concrete class nahi pata. ShapeFactory.create('circle'). Use: jab type runtime pe decide ho.

Q166. Observer pattern kya hai?

Ek subject ke change hone par uske saare subscribers (observers) automatically notify hote hain. Use: event systems, notifications, YouTube subscribe.

Q167. Strategy pattern kya hai?

Algorithms ko alag classes me rakho aur runtime pe swap karo. Parking fee (hourly/flat), payment (UPI/card). Open/Closed follow hota hai.

Q168. Singleton ka nuksaan?

Global state (testing mushkil), hidden dependencies, thread-safety issues, SRP violation ka risk.

Q169. Decorator, Adapter, Builder - ek line me?

Decorator: object me runtime pe naya behavior wrap karna. **Adapter:** incompatible interface ko compatible banana. **Builder:** complex object step-by-step banana (many optional fields).

PART 13

Predict the Output

Interview me code padhke output batana - pehle khud socho, fir answer padho

4-STEP METHOD: Trick: 1) Kaun sa object bana? 2) self kaun hai? 3) Kis class ka method chalega (MRO / override)? 4) Kya mutable object change ho raha hai?

Q170. PO-1: Method overriding

Answer: B, fir A. Child ka object apna overridden method chalata hai.

```
class A:
    def show(self): print('A')
class B(A):
    def show(self): print('B')
B().show()
A().show()
```

Q171. PO-2: Parent constructor chalega?

Answer: Sirf **C init**. Child ka constructor hai to parent ka auto nahi chalta.

```
class P:
    def __init__(self): print('P init')
class C(P):
    def __init__(self): print('C init')
C()
```

Q172. PO-3: Child me constructor nahi

Answer: P init - child me nahi mila to parent ka chal gaya.

```
class P:
    def __init__(self): print('P init')
class C(P): pass
C()
```

Q173. PO-4: super() ka order

Answer: C start, P, C end (control parent me jaata hai, fir wapas aata hai).

```
class P:
    def __init__(self): print('P')
class C(P):
    def __init__(self):
        print('C start')
        super().__init__()
        print('C end')
C()
```

Q174. PO-5: Private access

Answer: Pehla print **5** (mangled name), doosra **AttributeError**.

```
class A:
    def __init__(self): self.__x = 5
a = A()
print(a._A__x)
print(a.__x)
```

Q175. PO-6: Class variable shadow

Answer: 5 0 0. a.c = 5 ne a me naya instance variable banaya; class var nahi badla.

```
class T:
    c = 0
a = T(); b = T()
a.c = 5
print(a.c, b.c, T.c)
```

Q176. PO-7: Static counter

Answer: 3.

```
class T:
    n = 0
    def __init__(self):
        T.n += 1
T(); T(); T()
print(T.n)
```

Q177. PO-8: Mutable class variable (classic trap)

Answer: [1]. List class variable hai, dono objects share karte hain. Fix: self.items = [] constructor me.

```
class S:
    items = []
    def add(self, x): self.items.append(x)
a = S(); b = S()
a.add(1)
print(b.items)
```

Q178. PO-9: List ko function me pass

Answer: [1, 9] - mutable, reference se pass.

```
def f(l): l.append(9)
L = [1]
f(L)
print(L)
```

Q179. PO-10: Reassign vs mutate

Answer: [1]. Reassign sirf local reference badalta hai.

```
def f(l):
    l = [100]
L = [1]
f(L)
print(L)
```

Q180. PO-11: Tuple me add

Answer: (1, 2) - immutable, naya tuple local me bana.

```
def f(t): t += (4,)
T = (1, 2)
f(T)
print(T)
```

Q181. PO-12: Method overloading

Answer: 2 aur phir **TypeError** (missing argument x). Doosra f pehle ko overwrite kar deta hai.

```
class A:
    def f(self): print(1)
    def f(self, x): print(2)
A().f(5)
A().f()
```

Q182. PO-13: MRO

Answer: B aur **['D', 'B', 'C', 'A', 'object']**.

```
class A:
    def m(self): print('A')
class B(A):
    def m(self): print('B')
class C(A):
    def m(self): print('C')
class D(B, C): pass
D().m()
print([c.__name__ for c in D.__mro__])
```

Q183. PO-14: Recursion trap

Answer: RecursionError - self.m() khud ko hi call karta rahega.

```
class A:
    def m(self): return self.m()
A().m()
```

Q184. PO-15: __str__ vs list

Answer: S aur **[R]**. print(obj) __str__, container ke andar __repr__.

```
class P:
    def __str__(self): return 'S'
    def __repr__(self): return 'R'
print(P())
print([P()])
```

Q185. PO-16: == aur is

Answer: False False (default __eq__ identity check karta hai), fir **True**.

```
class P:
    def __init__(self, x): self.x = x
a = P(1); b = P(1)
print(a == b, a is b)
c = a
print(c is a)
```

Q186. PO-17: Fraction add without __add__

Answer: TypeError: unsupported operand type(s) for +: 'F' and 'F'.

```
class F:
    def __init__(self, n, d): self.n, self.d = n, d
print(F(1,2) + F(1,3))
```

Q187. PO-18: Private method call from child

Answer: AttributeError: 'C' object has no attribute '_C__hide' (name mangling).

```
class P:
    def __hide(self): print('hidden')
class C(P):
    def go(self): self.__hide()
C().go()
```

Q188. PO-19: super() me variable

Answer: AttributeError: 'super' object has no attribute 'x'. super() se variables nahi, methods milte hain. self.x chalta.

```
class P:
    def __init__(self): self.x = 1
class C(P):
    def __init__(self):
        super().__init__()
        print(super().x)
C()
```

Q189. PO-20: Abstract instantiate

Answer: TypeError: Can't instantiate abstract class S with abstract method f.

```
from abc import ABC, abstractmethod
class S(ABC):
    @abstractmethod
    def f(self): pass
S()
```

Q190. PO-21: Child abstract implement na kare

Answer: TypeError - child ne f implement nahi kiya to wo bhi abstract hi hai.

```
class S(ABC):
    @abstractmethod
    def f(self): pass
class C(S): pass
C()
```

Q191. PO-22: Shallow copy trap

Answer: [[1, 9], [2]] - nested list share hui.

```
import copy
a = [[1],[2]]
b = copy.copy(a)
b[0].append(9)
print(a)
```

Q192. PO-23: Mutable default argument

Answer: [1] aur **[1, 2]**. Default list ek hi baar bani aur share hoti hai. Fix: l=None, fir if l is None: l = [].

```
def f(x, l=[]):
    l.append(x)
    return l
print(f(1)); print(f(2))
```

Q193. PO-24: @property

Answer: AttributeError: can't set attribute (setter define nahi).

```
class A:
    def __init__(self): self._v = 1
    @property
    def v(self): return self._v
a = A()
a.v = 5
```

Q194. PO-25: Instance count

Answer: True - b = a se naya object nahi bana.

```
class T:
    def __init__(self):
        self.id = id(self)
a = T(); b = a
print(a.id == b.id)
```

PART 14

LLD Scenarios + Mini Project

Notes Part 3, 10 + Video Ch.21 (IRCTC project)

INTERVIEW FORMULA: LLD answer ka format: 1) Requirements puchho 2) Entities / classes identify karo 3) Relationships (is-a / has-a) 4) Patterns (Strategy, Factory, State) 5) Code skeleton 6) Edge cases / extensibility.

14.1 Parking Lot System

```
from abc import ABC, abstractmethod
class Vehicle(ABC):                          # abstraction
    def __init__(self, plate): self.plate = plate
    @abstractmethod
    def spot_type(self): pass
class Car(Vehicle):
    def spot_type(self): return 'MEDIUM'
class Bike(Vehicle):
    def spot_type(self): return 'SMALL'
class Truck(Vehicle):
    def spot_type(self): return 'LARGE'

class ParkingSpot:
    def __init__(self, id, kind): self.id, self.kind, self.__free = id, kind, True
    def is_free(self): return self.__free             # encapsulation
    def park(self): self.__free = False
    def leave(self): self.__free = True

class Level:
    def __init__(self, spots): self.spots = spots     # composition
    def find(self, kind):
        return next((s for s in self.spots if s.kind == kind and s.is_free()), None)

class FeeStrategy(ABC):                              # strategy pattern
    @abstractmethod
    def fee(self, hours): pass
class HourlyFee(FeeStrategy):
    def fee(self, hours): return hours * 20

class ParkingLot:
    def __init__(self, levels, strategy): self.levels, self.strategy = levels, strategy
    def park(self, vehicle):
        for lv in self.levels:
            spot = lv.find(vehicle.spot_type())
            if spot: spot.park(); return spot
        return None                                  # lot full
```

Q195. Parking Lot: kaun si OOP concepts use hoti hain?

Abstraction (Vehicle abstract), **Inheritance** (Car/Bike/Truck extends Vehicle), **Polymorphism** (spot_type, fee), **Encapsulation** (spot availability hidden), **Composition** (ParkingLot has Levels has Spots), **Strategy** (fee), **Factory** (vehicle creation), **Singleton** (ParkingLot).

Q196. Parking Lot: core entities?

ParkingLot, Level, ParkingSpot, Vehicle (abstract) -> Car, Bike, Truck, Ticket, Payment, EntryGate/ExitGate.

Q197. Parking Lot me concurrency (2 log same spot ek saath)?

Spot allocation ko **lock** (synchronized / threading.Lock) me rakho, ya atomic compare-and-set. Spot ka state encapsulated rakho.

14.2 ATM System (video ka project + notes)

Q198. ATM system design karo (classes, patterns).

Entities: **ATM, Card, Account, Transaction (abstract) -> Withdraw, Deposit, BalanceCheck**. **State Pattern** (Idle, CardInserted, PinVerified). Cash dispenser ke liye **Chain of Responsibility** (500 -> 200 -> 100 notes). **Encapsulation:** balance/PIN private, sirf methods se change. Video wale version me: pin aur balance private (__), getter/setter, deposit/withdraw/check_balance/create_pin, static counter se customer id.

Q199. ATM me withdraw me kaun si validation zaroori hai?

1) PIN verify, 2) amount <= balance (insufficient funds), 3) amount > 0 aur multiple of 100, 4) daily limit, 5) ATM me cash available. Alag check_pin() method banao taaki bar-bar same code repeat na ho (DRY).

14.3 Library, Elevator, Chess aur baaki LLDs

System	Entities	Patterns / OOP points
Library Management	Library, Book, BookItem (physical copy), Member, Librarian, Loan/Reservation, Fine	Member max N books (composition); Book base + Magazine/EBook (inheritance); Fine = Strategy; Search interface
Elevator	Elevator, Floor, Request, Controller/Scheduler	States: Idle, MovingUp, MovingDown (State); SCAN/nearest-car = Strategy; request queue alag
Chess	Game, Board, Cell, Player, Move, Piece (abstract) -> King, Queen, Rook, Bishop, Knight, Pawn	Har piece canMove() override (Polymorphism); move history Stack (undo)
Tic-Tac-Toe	Game, Board, Player, Symbol	Win-check O(1) with row/col counters; Strategy for AI player
BookMyShow	Movie, Theatre, Screen, Show, Seat, Booking, Payment	Seat lock (concurrency), Observer for notifications, Factory for payment
Splitwise	User, Group, Expense, Split (Equal/Exact/Percent), Balance	Strategy for split types; Observer for notifications
Vending Machine	Machine, Item, Coin, Inventory	State pattern (Idle, HasMoney, Dispensing)
Snake and Ladder	Game, Board, Dice, Player, Snake, Ladder	Dice strategy; Board composition

Q200. Library Management System ka design batao.

Book (title, author, ISBN) vs **BookItem** (physical copy, barcode, status). **Member** (max N books, composition of loans). Magazine/EBook inheritance se. **Fine calculation** Strategy se. Search interface (by title/author). Reservation aur notification Observer se.

Q201. Elevator system me scheduling kaise OOP se handle karoge?

Scheduler ko Strategy banao (SCAN, nearest car). Elevator ke states (Idle, MovingUp, MovingDown) State pattern se. Request queue alag class me, taaki naya algorithm bina Elevator class badle add ho (Open/Closed).

Q202. Chess me polymorphism kahan dikhta hai?

Piece abstract class me can_move(board, src, dest) abstract; King, Rook, Bishop apna rule override karte hain. Board bas piece.can_move() bulata hai - piece type ki parwah nahi.

14.4 Mini Project (video): IRCTC jaisa Train App

Idea: Console app jahan user menu se choose kare: 1) Live train status 2) PNR check 3) Train schedule 4) Trains at station 5) Trains between stations. Data kahan se? **API** (IndianRailAPI) se. **Tools:** PyCharm jaisa proper IDE (IDLE sirf padhane/quick test ke liye), pip install requests.

```
import requests

class IRCTC:
    def __init__(self):
        user = input('''Kya karna chahte ho?
        1. Live train status
        2. PNR check
        3. Train schedule''')
        if user == '3':
            self.train_schedule()

    def train_schedule(self):
        train_no = input('Train number daalo: ')
        self.fetch_data(train_no)

    def fetch_data(self, train_no):
        url = 'https://<indianrailapi>/TrainSchedule/apikey/<KEY>/TrainNumber/{}'.format(train_no)
        response = requests.get(url)          # server ko hit
        data = response.json()                # JSON -> Python dict
        for i in data['Route']:               # route = list of dicts
            print(i['StationName'], '|', i['ArrivalTime'],
                  '|', i['DepartureTime'], '|', i['Distance'], 'km')

obj = IRCTC()
```

Q203. API kya hota hai? Is project me kyun use kiya?

Application Programming Interface - ek function/service jo kisi aur ka data tumhe laake deta hai. Railway ka data hamare paas nahi hota; API (URL) pe sahi input (train number + API key) dene par wo data wapas deta hai.

Q204. JSON format kya hai? Python me kaise padhte hain?

Key-value data format jo Python dictionary jaisa dikhta hai. API response ko response.json() se dict me convert karte hain, fir data['Route'] jaisa keys se andar jaate hain. Padhne ke liye online JSON viewer use kar sakte ho.

Q205. requests library kya hai? Install kaise karte hain?

HTTP requests bhejne ke liye library: requests.get(url). Install: pip install requests. Code me import requests.

Q206. Bug: koi bhi train number dalo, hamesha ek hi train ka data aa raha. Kyun?

URL me train number **hardcode** tha. Fix: '...TrainNumber/{}'.format(train_no) se user ka input URL me daalo (string formatting).

Q207. Is project me OOP kahan lagi? Aur improvement?

Menu-driven **IRCTC class** (constructor me menu, alag methods har feature ke liye). Improvement: **Train class** (Station ke saath aggregation), API calls ko alag APIClient class, error handling (invalid train, limit exceed), inheritance se alag report types.

Q208. Free API account me limit khatam ho jaye to?

Free plan me daily request limit hoti hai (transcript me bhi limit cross hui). Ek din baad ya doosre account se try karo; production me paid plan ya caching use karo.

PART 15

Rapid Fire + Cheat Sheet + Checklist

Last-minute revision: interview se pehle bas ye padh lo

15.1 Rapid Fire (1-line answers)

#	Question	Answer
1	OOP ke 4 pillars?	Encapsulation, Abstraction, Inheritance, Polymorphism
2	Class kya hai?	Blueprint / apna data type (data + methods)
3	Object kya hai?	Class ka instance, memory leta hai
4	Python me constructor?	__init__ (object banate hi auto-call)
5	self kya hai?	Current object ka reference (explicit first param)
6	Method vs function?	Method class ke andar, function bahar
7	Magic method kya?	__x__ naam, Python auto-call karta hai
8	print(obj) kaun sa method?	__str__ (nahi to __repr__)
9	a + b kaun sa method?	__add__
10	Operator overloading?	Operator ko custom class ke liye naya behavior
11	Python me private?	__x (name mangling), true private nahi
12	Name mangling?	__x -> _Class__x
13	Encapsulation?	Data hide + controlled access (getter/setter)
14	Getter/Setter ka fayda?	Validation aur control
15	@property?	Method ko attribute ki tarah use karna
16	Static variable?	Class level, sabke liye shared
17	Instance variable?	Har object ka alag, constructor me self.x
18	@staticmethod?	Self/cls nahi, utility
19	@classmethod?	cls milta hai, alternate constructor
20	Reference variable?	Jisme object ka address hota hai
21	Pass by ___ in Python?	Object reference (mutable change dikhta hai)
22	Mutable types?	list, dict, set, user objects
23	Immutable types?	int, float, str, tuple, frozenset
24	Aggregation?	Has-A (Customer has Address), weak ownership
25	Composition?	Strong has-a, container ke saath contained destroy
26	Association?	General uses-a, independent
27	Inheritance?	Parent ki properties child ko milna
28	Kya inherit hota hai?	Data members, methods, constructor (private nahi)
29	Inheritance direction?	Upar se neeche (child parent se leta hai)
30	super()?	Parent method/constructor call (class ke andar hi)
31	Child me __init__ ho to?	Parent ka auto nahi chalta, super().__init__()
32	Types of inheritance?	Single, Multilevel, Hierarchical, Multiple, Hybrid
33	Java me multiple inheritance?	Classes se nahi, interfaces se
34	MRO?	Method search order, left-to-right (C3), Cls.__mro__
35	Diamond problem?	D(B,C), B,C(A) - ambiguity; Python MRO se, C++ virtual inh.
36	Method overriding?	Child me same signature, runtime, child ka chalta
37	Method overloading?	Same naam alag params; Python me native nahi

#	Question	Answer
38	Overloading Python me?	Default args, *args, singledispatch
39	Duck typing?	Type nahi behavior matter karta hai
40	Abstract class?	Object nahi ban sakta, abstract methods
41	ABC?	abc.ABC + @abstractmethod
42	Abstract vs Interface?	IS-A with state vs CAN-DO, multiple implement
43	Shallow vs Deep copy?	Nested references share vs fully independent
44	copy module?	copy.copy, copy.deepcopy
45	__new__ vs __init__?	Object banata vs initialize karta
46	Singleton?	Ek hi instance (__new__ / private ctor)
47	Factory?	Object creation logic ek jagah
48	Observer?	Subject change -> subscribers notify
49	Strategy?	Algorithm runtime pe swap
50	S in SOLID?	Single Responsibility
51	O in SOLID?	Open for extension, closed for modification
52	L in SOLID?	Liskov - child parent ki jagah chale
53	I in SOLID?	Chhote specific interfaces
54	D in SOLID?	Abstraction pe depend karo, concrete pe nahi
55	Composition over inheritance?	Has-A zyada flexible, loose coupling
56	Virtual destructor?	C++ me base ptr se delete par derived cleanup
57	Object slicing?	Derived ko base by value assign - extra part cut
58	is vs == ?	identity vs equality
59	__slots__?	__dict__ nahi, memory kam
60	dataclass?	__init__/__repr__/__eq__ auto
61	Immutable class?	final class, private final fields, no setters, deep clone
62	DRY?	Don't Repeat Yourself - inheritance ka reason
63	Early vs Late binding?	Compile-time vs runtime
64	Constructor return?	Sirf None
65	Abstract object banane par?	TypeError
66	super() class ke bahar?	Kaam nahi karta (AttributeError)

15.2 One-Page Cheat Sheet

```
class Demo(Parent):                  # inheritance
    count = 0                        # static / class variable

    def __init__(self, a):          # constructor
        super().__init__()           # parent ka constructor
        self.a = a                   # public instance var
        self._b = 1                  # protected (convention)
        self.__c = 2                 # private (name mangling)
        Demo.count += 1

    def __str__(self):  return f'Demo({self.a})'      # print()
    def __repr__(self): return f'Demo(a={self.a})'    # debug
    def __add__(self, o): return Demo(self.a + o.a)    # +
    def __eq__(self, o):  return self.a == o.a         # ==

    @property
    def c(self): return self.__c                       # getter
    @c.setter
    def c(self, v): self.__c = v                       # setter

    @staticmethod
    def util(): ...                                    # no self
    @classmethod
    def make(cls): return cls(0)                       # cls

    def work(self):                                    # overriding + super
        super().work()
```

Concept	Yaad rakhne ki trick
Encapsulation	CAPSULE - dawai andar band, cover (class) se bahar
Abstraction	CAR - steering use karo, engine ka kaam mat dekho
Inheritance	PAPA ki property beta ko - upar se neeche
Polymorphism	Ek naam, kai roop - 3 types: Override, Overload, Operator
Aggregation	HAS-A: Customer has Address (diamond whole ki taraf)
self	Jo object abhi kaam kar raha hai wahi self
Constructor	Auto button - user ko control nahi deta
Static	SHARED - sabke liye ek (counter, bank name)
MRO	LEFT to RIGHT, pehla mila wahi chalega
Mutable pass	Function ke andar change = bahar bhi change

15.3 Interview Prep Checklist

Done?	Topic
[ ]	4 pillars ek line me, har ek ka real-life + code example
[ ]	Class vs Object, Method vs Function, Static vs Instance
[ ]	__init__, self, magic methods (__str__, __repr__, __add__, __eq__)
[ ]	Private / name mangling / getter-setter / @property
[ ]	Mutable vs immutable aur pass-by-reference traps
[ ]	Inheritance types, super(), MRO, diamond problem
[ ]	Overriding vs overloading vs operator overloading (+ Python workaround)
[ ]	Abstract class vs interface, ABC
[ ]	Shallow vs deep copy, __new__ vs __init__
[ ]	SOLID ke 5 naam + example, Composition over Inheritance
[ ]	Singleton, Factory, Observer, Strategy - code karke dikha sako
[ ]	Ek LLD (Parking Lot) end-to-end explain karne ki practice
[ ]	Predict-the-output (Part 13) khud solve karo, bina answer dekhe
[ ]	Java/C++ differences: access modifiers, multiple inheritance, virtual destructor

FINAL TIP: Interview strategy: pehle definition (1 line), phir real-life example, phir tiny code, phir 'kab use karte hain / nuksaan'. Yeh 4-step answer interviewer ko impress karta hai.

Yaad rakho: Theory se zyada code likho. Video me bhi bola gaya hai - jab tak khud application nahi banate OOP samajh nahi aata. ATM, Fraction, IRCTC jaise chhote projects khud banao aur GitHub pe daalo. **All the best - ab tum OOPs me master ho!**